

Project: AutoTrack – Vehicle Service & Maintenance Management System

1. Introduction

This document outlines the Low-Level Design (LLD) for **AutoTrack**, a web-based system for automotive service centers. The system manages vehicle registration, service scheduling, maintenance history, spare parts inventory, and performance analytics for operational efficiency.

Supports backend development using **Java (Spring Boot)** and **.NET (ASP.NET Core)**.

2. Module Overview

- 2.1 Vehicle Registration & Customer Profile Module
- 2.2 Service Scheduling & Maintenance History Module
- 2.3 Spare Parts Inventory Management Module
- 2.4 Billing & Payment Processing Module
- 2.5 User Authentication & Role Management Module

3. Architecture Overview

3.1 Architectural Style

- **Frontend:** Angular/React for customer and admin dashboards
- **Backend:** REST APIs for service workflows and inventory management
- **Database:** MySQL/SQL Server for structured vehicle and service data

3.2 Component Interaction

- Frontend captures service requests and displays maintenance history
- Backend processes service schedules and inventory updates

4. Module-Wise Design

4.1 Vehicle Registration & Customer Profile Module

Features:

- Register vehicles and link to customer profiles
- View/update customer details

Entities:

- **Vehicle**
 - VehicleID
 - VIN

- CustomerID
- Model
- Year
- Status (ACTIVE, INACTIVE)
- **Customer**
 - CustomerID
 - Name
 - ContactInfo
 - Email

4.2 Service Scheduling & Maintenance History Module

Features:

- Schedule preventive and corrective maintenance
- Track service history and status

Entities:

- **ServiceRecord**
 - ServiceID
 - VehicleID
 - ServiceType
 - ScheduledDate
 - CompletionDate
 - Status (PENDING, COMPLETED)

4.3 Spare Parts Inventory Management Module

Features:

- Manage spare parts stock and reorder levels
- Link parts to service records

Entities:

- **SparePart**
 - PartID
 - Name
 - QuantityAvailable
 - ReorderLevel

4.4 Billing & Payment Processing Module

Features:

- Generate invoices for completed services

- Track payment status

Entities:

- **Invoice**
 - InvoiceID
 - ServiceID
 - Amount
 - PaymentStatus (PAID, UNPAID)

4.5 User Authentication & Role Management Module

Features:

- Secure login and role-based access for customers and staff

Entities:

- **User**
 - UserID
 - Username
 - Password
 - Role (CUSTOMER, SERVICE_AGENT, ADMIN)

5. Deployment Strategy

- **Local:** Developer machines with sample vehicle and service data
- **Production:** Cloud deployment with secure APIs and role-based access

6. Database Design

Tables: Customer → Vehicle → ServiceRecord → SparePart → Invoice → User

7. User Interface Design

Wireframes:

- **Customer Portal:** View vehicle details, book service
- **Admin Dashboard:** Manage inventory, monitor service schedules

8. Non-Functional Requirements

- **Performance:** Handle 10,000 service requests per day
- **Security:** Encrypted data, compliance with automotive service standards
- **Scalability:** Support multiple service centers

9. Assumptions and Constraints

- Customers book services via web portal
- Initial rollout for passenger vehicles only